When executing our test case using two JPQL queries, we get the following results based on the underlying plan cache size:

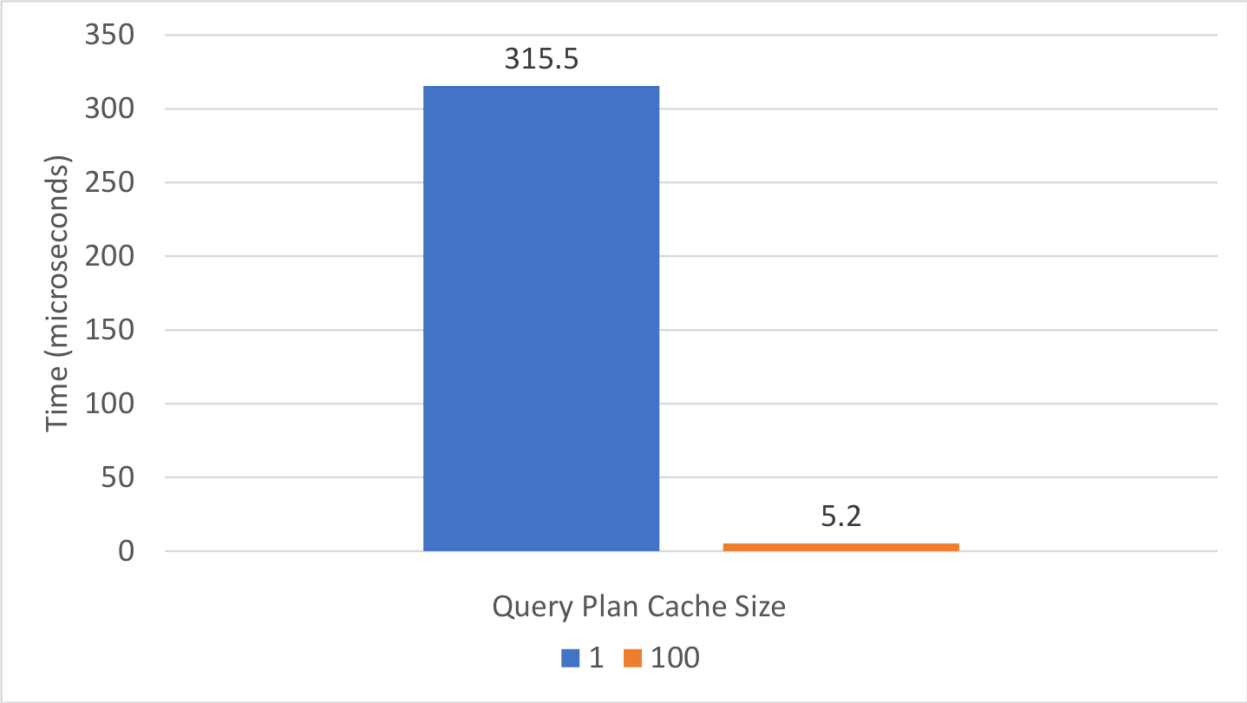

Figure 14.6: Entity query plan cache performance gain

When the plan cache size is 100, both entity queries will be compiled just once since any successive query execution will fetch the previously compiled plan from the cache, therefore speeding up the `Query` object creation.

On the other hand, when the query plan cache size is 1, the entity queries are compiled on every execution, hence the Query‘ object creation will take way longer this time.

Only for entity queries, the plan cache can really make a difference in terms of performance. For native queries, the gain is less significant.

For entity queries (JPQL and Criteria API), it is important to set the `hibernate.query.plan_cache_max_size` property so that it can accommodate all queries being executed. Otherwise, some entity queries might have to be recompiled, therefore increasing the transaction response time.

For native SQL queries, the `ParameterMetadata` cache can also provide a performance improvement, although not as significant as for entity queries:

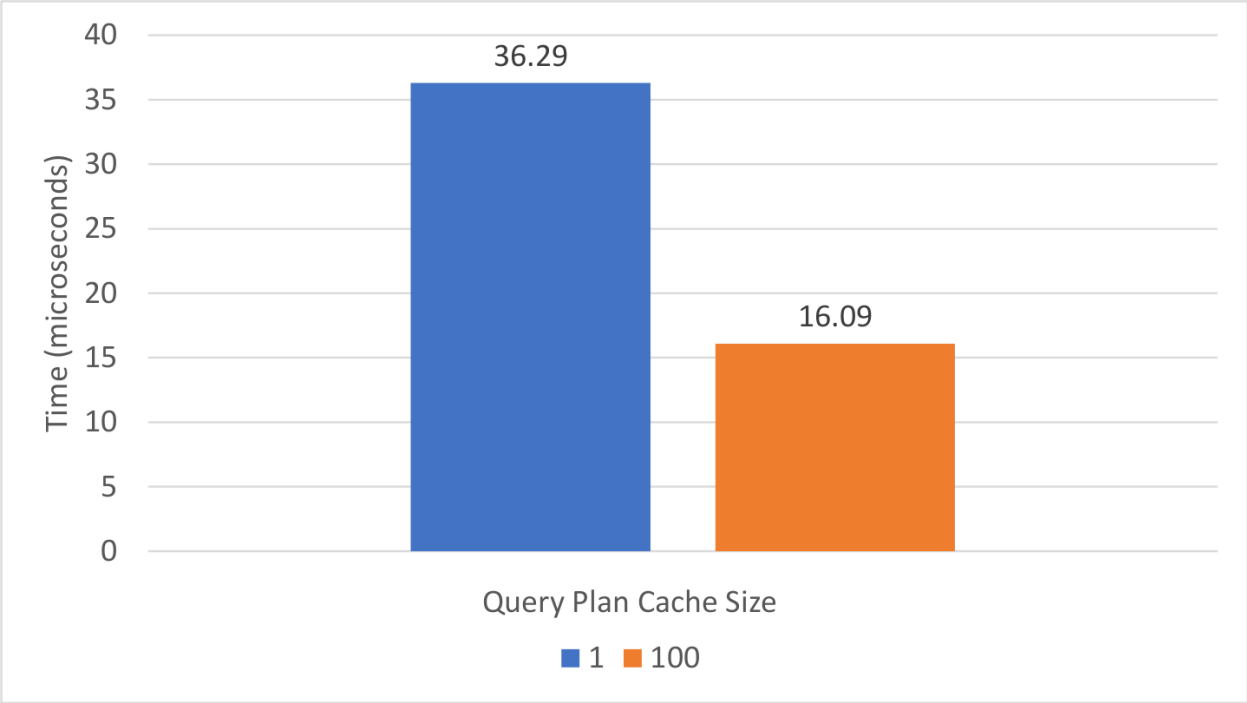

Figure 14.7: Native SQL query plan cache performance gain

15. Caching

15.1 Caching flavors

Caching is everywhere. For instance, the CPU has several caching layers to decrease the latency associated with accessing data from the main memory. Being close to the processing unit, the CPU cache is very fast. However, compared to the main memory, the CPU cache is very small and can only store frequently-accessed data.

To speed up reading and writing to the underlying disk drive, the operating system uses caching as well. Data is read in pages that are cached into main memory, so frequently-accessed data is served from OS buffers rather than the disk drive. Disk cache improves the write operations as well because modifications can be buffered and flushed at once, therefore improving write throughput.

Since indexes and data blocks are better off served from memory, most relational database systems employ an internal caching layer.

So even without explicitly setting up a caching solution, an enterprise application already uses several caching layers. Nevertheless, enterprise caching is most often a necessity, and there are several solutions that can be used for this purpose.

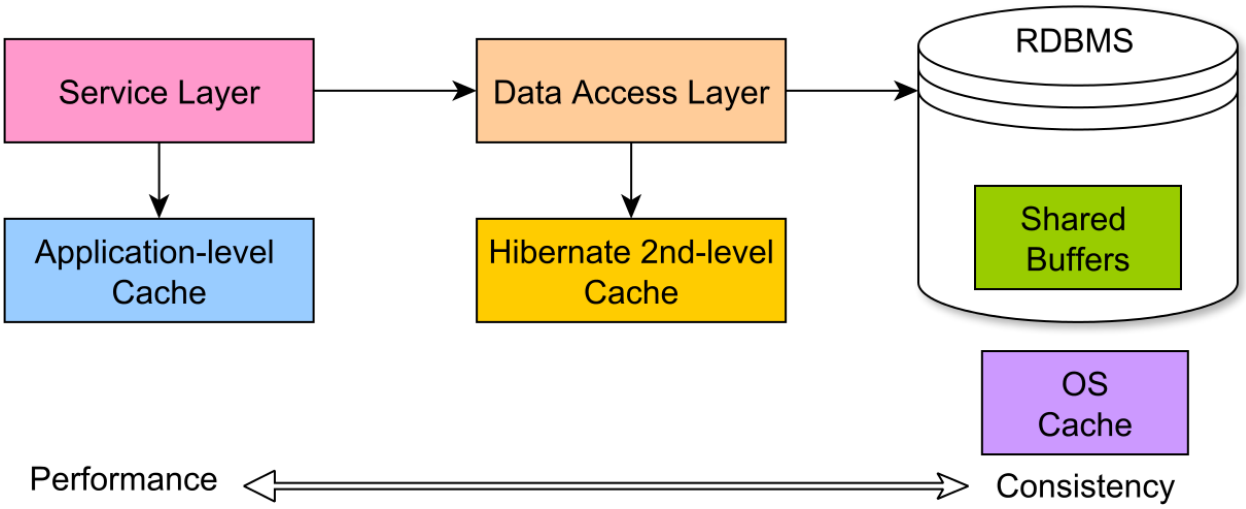

Figure 15.1: Enterprise caching layers

As illustrated in the diagram above, caching entails a trade-off. On one hand, bypassing the underlying data access layer can speed up reads and writes. However, the farther the caching solution is situated, the more difficult it is for it to maintain consistency with the underlying database system.

For it guarantees ACID transactions, the database cache is highly consistent, so, from a data integrity perspective, it entails no risk of reading stale data. However, the database engine can only spare disk access, and so it cannot alleviate the networking overhead. More, if the data access layer needs to fetch an aggregate that spans over multiple database tables, the result set would either contain many joins, or it will require multiple secondary queries. The more complex the data access pattern, the more work a database server has to do, and, for this reason, it is common to use an application-level cache as well.

Most often, application-level caches are key-value stores. Once an aggregate is fetched from the database, it can be stored in the application cache so that any successive request can bypass the database entirely. The application-level cache can outperform the database engine because it can bypass the networking overhead associated with fetching result sets.

Another very important reason for using an application-level caching solution is that it can provide a safety hook for when the database has to be taken down for maintenance.

If the front-end cache stores a sufficient amount of data, it can serve as a temporary replacement for the database system, allowing read-only operations to be served from the cache. Even if write operations are prevented while the database system is unavailable, the read-only mode increases the overall system availability.

However, application-level caches come at a price, and ensuring consistent reads is no longer a trivial thing to do. Because of its tight integration with Hibernate, the second-level cache can avoid many consistency-related issues associated with application-level caches.

The Hibernate second-level cache is a data access caching solution that aims to reduce database load when it comes to fetching entities. Along with it collection cache component, the second-level cache allows retrieving an entire entity graph without a single access to the database.

As explained in the previous chapter, fetching entities is usually associated with propagating entity state transitions. Therefore, the second-level cache can improve response time for read-write transactions without compromising data consistency.

Application-level caches are useful for read scenarios, while the second-level cache consistency guarantee is better suited for offloading write traffic.

15.2 Cache synchronization strategies

In database nomenclature, the system of record represents the source of truth when information is scattered among various data providers. Duplicating data, so that it resides closer to application layers, can improve response time at the price of making it more difficult to synchronize the two data copies. To avoid inconsistent reads and data integrity issues, whenever a change occurs in the system, it is very important to synchronize both the database and the cache.

There are various ways to keep the cache and the underlying database in sync, and this section is going to present some of the most common cache synchronization strategies.

15.2.1 Cache-aside

The application code manually manages both the database system and the caching layer.

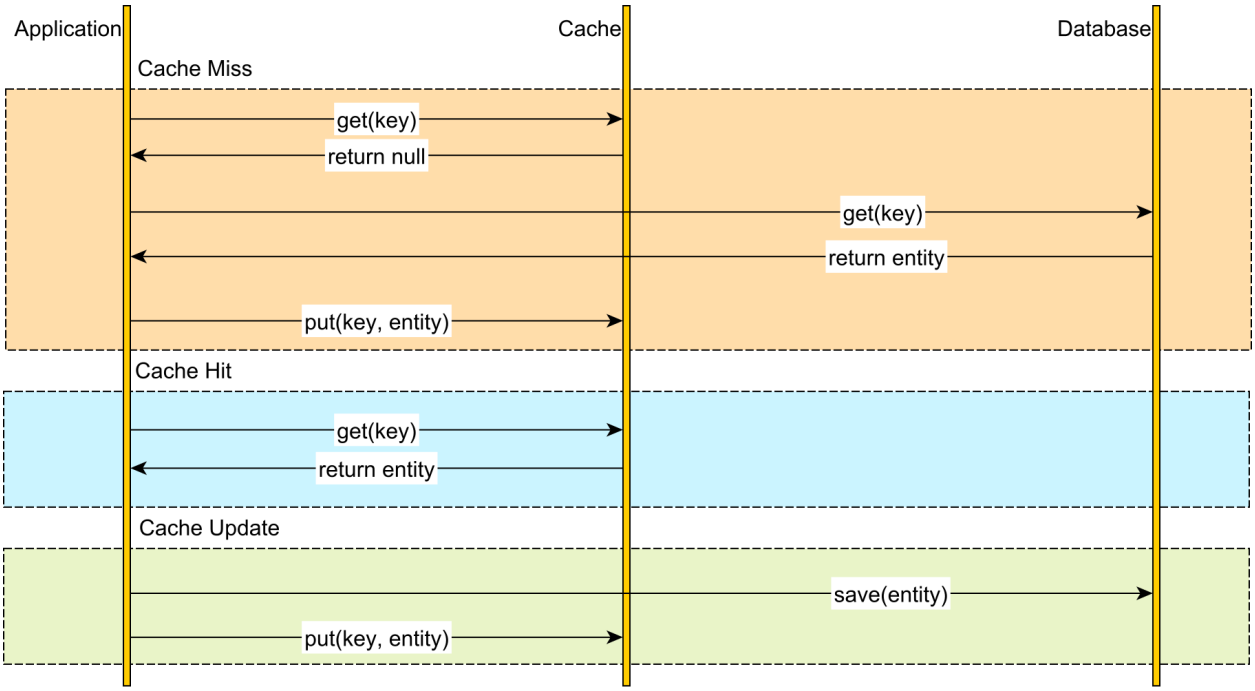

Figure 15.2: Cache-aside synchronization strategy

Before hitting the database, the application logic inspects the cache to see if the requested entity was previously loaded. Whenever an entity changes, the application must update both the database and the cache store.

Mixing application logic with caching management semantics breaks the Single Responsibility Principle. For this reason, it is good practice to move the caching logic into an AOP (aspect-oriented programming) interceptor, therefore decoupling the cache management logic from the business logic code.